

RAMDEOBABA UNIVERSITY NAGPUR



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Student Name :- Advait Kawale

Section :- A4 (B3)

Roll No :- 42

Course Name :- Design and Analysis of Algorithms

Course Code :- 24CS01TH0201

Title :- Practical 6

Practical 6

Aim :- Construction of OBST

Task 1 :-

Scenario :-

A university digital library system stores frequently accessed books using a binary search mechanism. The library admin wants to minimize the average search time for book lookups by arranging the book IDs optimally in a binary search tree.

Each book ID has a probability of being searched successfully and an associated probability for unsuccessful searches (when a book ID does not exist between two keys).

Your task is to determine the minimum expected cost of searching using an Optimal Binary Search Tree (OBST).

Ans)

Input)

```
#include <stdio.h>
#define MAX 100
#define INF 999999.0
int main()
{
    int n=4;
    int keys[]={0,10,20,30,40};
    double p[]={0,0.1,0.2,0.4,0.3};
    double q[]={0.05,0.1,0.05,0.05,0.1};
    double e[MAX+1][MAX+1],w[MAX+1][MAX+1];
    for(int i=1;i<=n+1;i++)
    {
        e[i][i-1]=q[i-1];
        w[i][i-1]=q[i-1];
    }
    for(int l=1;l<=n;l++)
    {
        for(int i=1;i<=n-l+1;i++)
        {
```

```

    int j=i+l-1;
    e[i][j]=INF;
    w[i][j]=w[i][j-1]+p[j]+q[j];
    for(int r=i;r<=j;r++)
    {
        double t=e[i][r-1]+e[r+1][j]+w[i][j];
        if(t<e[i][j])
        {
            e[i][j]=t;
        }
    }
}
}
printf("Minimum expected cost = %.4lf\n",e[1][n]);
}

```

```

1  #include <stdio.h>
2  #define MAX 100
3  #define INF 999999.0
4  int main()
5  {
6      int n=4;
7      int keys[]={0,10,20,30,40};
8      double p[]={0,0.1,0.2,0.4,0.3};
9      double q[]={0.05,0.1,0.05,0.05,0.1};
10     double e[MAX+1][MAX+1],w[MAX+1][MAX+1];
11     for(int i=1;i<=n+1;i++)
12     {
13         e[i][i-1]=q[i-1];
14         w[i][i-1]=q[i-1];
15     }
16     for(int l=1;l<=n;l++)
17     {
18         for(int i=1;i<=n-l+1;i++)
19         {
20             int j=i+l-1;
21             e[i][j]=INF;
22             w[i][j]=w[i][j-1]+p[j]+q[j];
23             for(int r=i;r<=j;r++)
24             {
25                 double t=e[i][r-1]+e[r+1][j]+w[i][j];
26                 if(t<e[i][j])
27                 {
28                     e[i][j]=t;
29                 }
30             }
31         }
32     }
33     printf("Minimum expected cost = %.4lf\n",e[1][n]);
34 }

```

Output)

Output

Minimum expected cost = 2.9000

Task 2 :- GFG

Given a sorted array `keys[0.. n-1]` of search keys and an array `freq[0.. n-1]` of frequency counts, where `freq[i]` is the number of searches to `keys[i]`. Construct a binary search tree of all keys such that the total cost of all the searches is as small as possible.

Let us first define the cost of a BST. The cost of a BST node is level of that node multiplied by its frequency. Level of root is 1.

Ans)

Input)

class Solution

```
{
    static int optimalSearchTree(int keys[],int freq[],int n)
    {
        int[][] dp=new int[n][n];
        int[] sumFreq=new int[n+1];
        for(int i=0;i<n;i++)
        {
            sumFreq[i+1]=sumFreq[i]+freq[i];
        }
        for(int length=1;length<=n;length++)
        {
            for(int i=0;i<=n-length;i++)
            {
                int j=i+length-1;
                dp[i][j]=Integer.MAX_VALUE;
                int sum=sumFreq[j+1]-sumFreq[i];
                for(int r=i;r<=j;r++)
                {
                    int cost=(r>i?dp[i][r-1]:0)+(r<j?dp[r+1][j]:0)+sum;
                    dp[i][j]=Math.min(dp[i][j],cost);
                }
            }
        }
        return dp[0][n-1];
    }
}
```

```

Java (21) [Start Timer]
1 class Solution
2 {
3     static int optimalSearchTree(int keys[],int freq[],int n)
4     {
5         int[][] dp=new int[n][n];
6         int[] sumFreq=new int[n+1];
7         for(int i=0;i<n;i++)
8         {
9             sumFreq[i+1]=sumFreq[i]+freq[i];
10        }
11        for(int length=1;length<=n;length++)
12        {
13            for(int i=0;i<=n-length;i++)
14            {
15                int j=i+length-1;
16                dp[i][j]=Integer.MAX_VALUE;
17                int sum=sumFreq[j+1]-sumFreq[i];
18                for(int r=i;r<=j;r++)
19                {
20                    int cost=(r>i?dp[i][r-1]:0)+(r<j?dp[r+1][j]:0)+sum;
21                    dp[i][j]=Math.min(dp[i][j],cost);
22                }
23            }
24        }
25        return dp[0][n-1];
26    }
27 }

```

Output)

[Courses](#)
[Tutorials](#)
[Practice](#)
[Jobs](#)

[Problem](#)
[Editorial](#)
[Submissions](#)
[Comments](#)

Output Window

Compilation Results

Custom Input

Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

104 / 104

Attempts: Correct / Total

1 / 1

Accuracy: 100%

Points Scored

8 / 8

Time Taken

0.29

Your Total Score: 8

Solve Next

Fixing Two nodes of a BST

Strictly Increasing Array

Word Wrap

Stay Ahead With:

Build 21 Projects in 21 Days
 Build real-world ML, Deep Learning & Gen AI projects

Java (21) [Start Timer]

```

1 class Solution
2 {
3     static int optimalSearchTree(int keys[],int freq[],int n)
4     {
5         int[][] dp=new int[n][n];
6         int[] sumFreq=new int[n+1];
7         for(int i=0;i<n;i++)
8         {
9             sumFreq[i+1]=sumFreq[i]+freq[i];
10        }
11        for(int length=1;length<=n;length++)
12        {
13            for(int i=0;i<=n-length;i++)
14            {
15                int j=i+length-1;
16                dp[i][j]=Integer.MAX_VALUE;
17                int sum=sumFreq[j+1]-sumFreq[i];
18                for(int r=i;r<=j;r++)
19                {
20                    int cost=(r>i?dp[i][r-1]:0)+(r<j?dp[r+1][j]:0)+sum;
21                    dp[i][j]=Math.min(dp[i][j],cost);
22                }
23            }
24        }
25        return dp[0][n-1];
26    }
27 }

```

Custom Input

Compile & Run

Submit